```
pi = "pi = π"
pi.encode("iso-8859-1")
```

produces:

```
        from prog.rb:2:in `<main>'
prog.rb:2:in `encode': U+03C0 from UTF-8 to ISO-8859-1
(Encoding::UndefinedConversionError)
```

You can, however, override the exceptional behavior, for example supplying a placeholder character to use when no direct translation is possible.

```
pi = "pi = π"
puts pi.encode("iso-8859-1", undef: :replace, replace: "??")
```

produces:

```
pi = ??
```

Sometimes you'll have a string containing binary data and you want that data to be interpreted as if it had a particular encoding. You can't use the encode method for this, because you don't want to change the byte contents of the string—you're just changing the encoding associated with those bytes. Use the force_encoding method to do this:

```
# encoding: ascii-8bit
str = "\xc3\xa9"     # e-acute in UTF-8
str.encoding   # => #<Encoding:ASCII-8BIT>
str.force_encoding("utf-8")
str.bytes.to_a # => [195, 169]
str.encoding   # => #<Encoding:UTF-8>
```

Finally, you can use encode (with two parameters) to convert between two encodings if your source string is ASCII-8BIT. This might happen if, for example, you're reading data in binary mode from a file and choose not to encode it at the time you read it. Here we fake that out by creating an ASCII-8BIT string that contains an ISO-8859-1 sequence (our old friend *olé*). We then convert the string to UTF-8. To do this, we have to tell encode the actual encoding of the bytes by passing it a second parameter:

```
# encoding: ascii-8bit
original = "ol\xe9"     # e-acute in ISO-8859-1
original.bytes.to_a # => [111, 108, 233]
original.encoding   # => #<Encoding:ASCII-8BIT>
new = original.encode("utf-8", "iso-8859-1")
new.bytes.to_a      # => [111, 108, 195, 169]
new.encoding        # => #<Encoding:UTF-8>
```

Symbols

Symbols don't have a lot of methods in Ruby. We've seen to_proc as a shortcut for creating a block. The to_proc method creates a block that is effectively equivalent to { |receiver, ...| receiver.send(symbol, ...)}. You usually see it used implicitly with &, but it can be used explicitly, too:

```
proc = :split.to_proc
proc.call("The pickaxe book")       # => ["The", "pickaxe", "book"]
proc.call("The pickaxe book", "e")  # => ["Th", " pickax", " book"]
```